algo – Team Reference Document

Содержание

1 Templates **2**
1.1 alloc.cpp . . . 2
1.2 tmp.cpp . . . 2
1.3 tmpFast.cpp . . . 2
1.4 cmp.sh . . . 2

2 Data Structures **2**
2.1 Disjoint Sparse Table . . . 2
2.2 Fenwick Tree . . . 3
2.3 Link-Cut Tree . . . 3
2.4 Node . . . 3
2.5 Persistent Treap . . . 4
2.6 Segment Tree . . . 5
2.7 Fast Segment Tree . . . 6
2.8 Splay Tree . . . 7
2.9 Sparse Table . . . 8
2.10 Treap . . . 8

3 Geometry **9**
3.1 Graham Scan . . . 9
3.2 Jarvis March . . . 9
3.3 2D Geometry . . . 10

4 Graphs **11**
4.1 Centroid Decomposition . . . 11
4.2 Cut Points . . . 11
4.3 Heavy-Light Decomposition . . . 12

5 Optimizations and Transforms **12**
5.1 Convex Hull Trick . . . 12
5.2 Fast Fourier Transform . . . 12
5.3 Compiler Optimizations . . . 12
5.4 XOR Convolution . . . 13

6 Strings **13**
6.1 String Algorithms . . . 13

1 Templates

1.1 alloc.cpp

```
#include "tmp.cpp"

const int MAX_MEM = 1e8;
int mpos = 0;
alignas(long long) char mem[MAX_MEM];

inline void * operator new(size_t n) {
        if (n & 7) n += 8 - (n & 7);
        char *res = mem + mpos;
        mpos += n;
        return (void *)res;
}

void operator delete(void *) {}
```

1.2 tmp.cpp

```
#include <bits/stdc++.h>

// #pragma GCC optimize("O3,unroll-loops")
// #pragma GCC target("avx2")

#define all(x) x.begin(), x.end()
#define rall(x) x.rbegin(), x.rend()
// #define int long long

using namespace std;
using ll = long long;
using ull = unsigned long long;
using ld = long double;
using pii = pair<int, int>;
template<typename T1, typename T2>
using pr = pair<T1, T2>;
using vi = vector<int>;
using vb = vector<bool>;
using vii = vector<pair<int, int>>;
using vll = vector<long long>;
template<typename T>
using ve = vector<T>;

const long long MOD = 1e9 + 7;
const int INF = 1e9 + 1;
const long long INFLL = 1e18 + 1;

inline void fchmin(int &a, int b) { a = min(a, b); }
inline void fchmax(int &a, int b) { a = max(a, b); }

template<typename T>
inline bool chmin(T &a, T b) {
        if (a > b) {
                a = b;
                return true;
        }
        return false;
}

template<typename T>
inline bool chmax(T &a, T b) {
        if (a < b) {
                a = b;
                return true;
        }
        return false;
}

template <typename... Args>
void dbg(const string& s, const Args&... args) {
        #ifdef DEBUG
        cout << s << ": ";
        ((cout << args << ' '), ...);
        cout << '\n';
        #endif
}

int reps = 1;

void solve() {

}

signed main() {
        ios::sync_with_stdio(0);
        cin.tie(0);

        #ifdef LOCAL
        freopen("input.txt", "r", stdin);
        freopen("output.txt", "w", stdout);
        #endif
        cin >> reps;

        for (int i = 1; i <= reps; i++) {
                #ifdef LOCAL
                cout << "Test #" << i << " ================\n"
    ;
                #endif
                solve();
                #ifdef LOCAL
                cout << "End test ==============\n\n";
                #endif
        }
        #ifdef LOCAL
        cout << "Runtime is: " << clock() * 1.0 /
    CLOCKS_PER_SEC << endl;
        #endif
}
```

1.3 tmpFast.cpp

```
#include <bits/stdc++.h>

// #pragma GCC optimize("O3,unroll-loops")
// #pragma GCC target("avx2")

#define all(x) x.begin(), x.end()
#define rall(x) x.rbegin(), x.rend()
// #define int long long

using namespace std;
using ll = long long;
using ull = unsigned long long;
using ld = long double;
using pii = pair<int, int>;
template<typename T1, typename T2>
using pr = pair<T1, T2>;
using vi = vector<int>;
using vb = vector<bool>;
using vii = vector<pair<int, int>>;
using vll = vector<long long>;
template<typename T>
using ve = vector<T>;

const long long MOD = 1e9 + 7;
const int INF = 1e9 + 1;
const long long INFLL = 1e18 + 1;

signed main() {
        ios::sync_with_stdio(0);
        cin.tie(0);

        #ifdef LOCAL
        freopen("input.txt", "r", stdin);
        freopen("output.txt", "w", stdout);
        #endif

        return 0;
}
```

1.4 cmp.sh

```
#!/bin/bash

a=$1
/opt/homebrew/bin/g++-15 -D_Alignof=__alignof__ -std=c++23 -
    Wall -Wextra -Wshadow -g -DLOCAL -D_GLIBCXX_DEBUG -O2 "$a
    .cpp" -o "$a"
```

2 Data Structures

2.1 Disjoint Sparse Table

```
#include "tmp.cpp"

struct DST {
        struct Node {
                int x;
                Node() : x(INF) {}
                Node(int _x) : x(_x) {}
        };

        Node merge(const Node &l, const Node &r) {
                return (l.x <= r.x) ? l : r;
        }

        int maxn, logn;
        ve<ve<Node>> dst;

        DST() {}
        DST(vi &a, int lg) : maxn(a.size()), logn(lg) {
                dst.resize(logn, ve<Node>(maxn + 1));
                int h = 32 - __builtin_clz(maxn);
                for (int d = 0; d < h; d++) {
                        int len = (1LL << d);
                        for (int mid = len; mid < maxn + len;
    mid += 2 * len) {
                                if (mid <= maxn) dst[d][mid] =
     Node();
                                for (int i = mid + 1; i <= min
    (mid + len, maxn); i++) {
                                        dst[d][i] = merge(dst[
    d][i - 1], Node(a[i - 1]));
                                }
```

```
                        for (int i = min(maxn - 1, mid
     - 1); i >= mid - len; i--) {
                                dst[d][i] = merge(dst[
    d][i + 1], Node(a[i]));
                        }
                }
        }

        Node get(int l, int r) {  // [l, r)
                int fbit = 31 - __builtin_clz(r ^ l);
                return merge(dst[fbit][l], dst[fbit][r]);
        }
};
```

2.2 Fenwick Tree

```
#include "tmp.cpp"

template<typename T>
struct FenwickTree {
        int maxn;
        ve<T> ft;

        FenwickTree() {}
        FenwickTree(int n) : maxn(n) {
                ft.resize(maxn);
        }

        void build(int *s) {
                for (int i = 0; i < maxn; i++) {
                        ft[i] += s[i];
                        int j = i | (i + 1);
                        if (j < maxn) ft[j] += ft[i];
                }
        }

        void update(int i, T x) {
                for (; i < maxn; i |= (i + 1)) ft[i] += x;
        }

        T get(int ql, int qr) {
                return get(qr - 1) - get(ql - 1);
        }

        T get(int r) {
                fchmin(r, maxn - 1);
                T res = 0;
                for (; r >= 0; r = (r & (r + 1)) - 1) res +=
    ft[r];
                return res;
        }

        int lower_bound(int pos) {
                int k = 0;
                for (int l = 29; l >= 0; l--) {
                        if (k + (1 << l) <= maxn && ft[k + (1
    << l) - 1] <= pos) {
                                k += (1 << l);
                                pos -= ft[k - 1];
                        }
                }
                return k;
        }

        int descent() {
                int k = 0;
                for (int l = 29; l >= 0; l--) {
                        if (k + (1 << l) <= maxn && ft[k + (1
     << l) - 1] == (1 << l)) {
                                k += (1 << l);
                        }
                }
                return k;
        }
};
```

2.3 Link-Cut Tree

```
#include "tmp.cpp"
#include "splayTree.cpp"

class LinkCutTree {
public:
        int n;
        ve<Node *> nodes;

        LinkCutTree() : n(0) {}

        LinkCutTree(int _n) : n(_n) {
                nodes.resize(n);
                for (int i = 0; i < n; i++) {
                        nodes[i] = new Node(i);
                }
        }

        int addNode() {
                int id = nodes.size();
                nodes.push_back(new Node(id));
                return id;
        }

        void makeRoot(Node *v) {
                if (!v) return;
                expose(v);
                v->reverse();
        }

        void expose(Node *v) {
                Node *u = v;
                Node *lst = nullptr;
                while (u != nullptr) {
                        splay(u);
                        u->push();
                        u->r = lst;
                        u->pull();
                        lst = u;
                        u = u->p;
                }
                splay(v);
        }

        bool link(int a, int b) {
                Node *v = nodes[a];
                Node *u = nodes[b];
                makeRoot(v);
                expose(u);
                if (v->p != nullptr) {
                        return false;
                }
                v->p = u;
                // u->pull();
                return true;
        }

        bool cut(int a, int b) {
                Node *v = nodes[a];
                Node *u = nodes[b];
                expose(u);
                splay(v);
                if (v->p != u) {
                        swap(u, v);
                        expose(u);
                        splay(v);
                        if (v->p != u) {
                                return false;
                        }
                }
                Node *lst = v->p;
                v->p = nullptr;
                if (lst) lst->pull();
                return true;
        }

        Node * getL(Node *v) {
                v->push();
                if (!v->l) return v;
                return getL(v->l);
        }

        int getRoot(int a) {
                Node *v = nodes[a];
                expose(v);
                return getL(v)->x;
        }

        int pathLen(int a, int b) {
                if (getRoot(a) != getRoot(b)) return -1;
                Node *u = nodes[a];
                Node *v = nodes[b];
                makeRoot(u);
                expose(v);
                return v->sz - 1;
        }
};
```

2.4 Node

```
class Node {
public:
        Node *l, *r;
        Node *p;
        int sz;
        int x;
        int y;
        bool rev;
        int summ;

        Node() {
                l = nullptr;
                r = nullptr;
                p = nullptr;
                sz = 0;
                x = 0;
                rev = false;
        }

        Node(int x) : x(x), sz(1) {
```

```
                l = nullptr;
                r = nullptr;
                p = nullptr;
                rev = false;
        }

        Node(int x, Node *_l, Node *_r) : x(x), sz(1) {
                l = _l;
                r = _r;
                if (_l) _l->p = this;
                if (_r) _r->p = this;
                p = nullptr;
                rev = false;
        }

        void reverse() {
                rev ^= 1;
                swap(l, r);
        }

        void push() {
                if (rev) {
                        if (l) l->reverse();
                        if (r) r->reverse();
                        rev = false;
                }
        }

        void pull() {
                sz = 1;
                if (l) sz += l->sz;
                if (r) sz += r->sz;
        }
};

int size(Node *v) {
        return (v) ? v->sz : 0;
}
```

2.5 Persistent Treap

```
#include "tmp.cpp"

mt19937 rng(1543);

class Node {
public:
        Node *l, *r;
        int sz;
        int x;
        // int summ;
        // bool reversed;

        Node() {
                // summ = 0;
                // reversed = false;
                sz = 1;
                x = 0;
                l = nullptr;
                r = nullptr;
        }

        Node(int _x) {
                // reversed = false;
                sz = 1;
                // summ = _x;
                x = _x;
                l = nullptr;
                r = nullptr;
        }

        void push() {
                return;
                // if (reversed) {
                //         if (l) l->reverse();
                //         if (r) r->reverse();
                //         reversed = false;
                // }
        }

        void pull() {
                // summ = x;
                // if (l) summ += l->summ;
                // if (r) summ += r->summ;
                sz = 1;
                if (l) sz += l->sz;
                if (r) sz += r->sz;
        }

        // void reverse() {
        //         reversed ^= 1;
        //         swap(l, r);
        // }
};

int _size(Node *t) {
        return (t) ? t->sz : 0;
}

// int _summ(Node *t) {
//         return (t) ? t->summ : 0;
// }

namespace Pivo {

pr<Node*, Node*> split(Node *t, int k) {  // <= >
        if (!t) return {nullptr, nullptr};
        t->push();
        if (_size(t) <= k) return {t, nullptr};
        if (k == 0) return {nullptr, t};
        t = new Node(*t);
        if (_size(t->l) >= k) {
                auto [t1, t2] = split(t->l, k);
                t->l = t2;
                t->pull();
                return {t1, t};
        } else {
                auto [t1, t2] = split(t->r, k - _size(t->l) -
    1);
                t->r = t1;
                t->pull();
                return {t, t2};
        }
}

Node * merge(Node *t1, Node *t2) {
        if (!t1) return t2;
        if (!t2) return t1;
        t1->push();
        t2->push();
        if (rand() % (_size(t1) + _size(t2)) < _size(t1)) {
                t1 = new Node(*t1);
                t1->r = merge(t1->r, t2);
                t1->pull();
                return t1;
        } else {
                t2 = new Node(*t2);
                t2->l = merge(t1, t2->l);
                t2->pull();
                return t2;
        }
}

Node *root = nullptr;

void insert(int x, int pos) {
        auto [t1, t2] = split(root, pos);
        root = merge(merge(t1, new Node(x)), t2);
}

void push_back(int x) {
        root = merge(root, new Node(x));
}

void remove(int pos) {
        auto [t1, t2] = split(root, pos);
        auto [t11, t12] = split(t2, 1);
        root = merge(t1, t12);
        delete t11;
}

int getF(Node *t) {
        while (t->l) {
                t = t->l;
        }
        return t->x;
}

// int getSumm(int l, int r) {
//         auto [t1, t2] = split(root, r);
//         auto [t11, t12] = split(t1, l);
//         if (!t12) return 0;
//         int res = t12->summ;
//         root = merge(merge(t11, t12), t2);
//         return res;
// }

// void assign(long long x, int l, int r) {
//         auto [t1, t2] = split(root, r);
//         auto [t11, t12] = split(t1, l);
//         t12->hasLower = false; t12->hasUpper = false;
//         t12->mn = x; t12->mx = x;
//         t12->summ = x * t12->sz; t12->cur = x;
//         t12->pushSet = true; t12->sp = x;
//         t12->fstel = x; t12->lstel = x;
//         t12->sp2 = 0;
//         t1 = merge(t11, t12);
//         root = merge(t1, t2);
// }

// void add(long long x, int l, int r) {
//         auto [t1, t2] = split(root, r);
//         auto [t11, t12] = split(t1, l);
//         t12->mn += x; t12->mx += x;
//         t12->summ += x * t12->sz; t12->cur += x;
//         t12->sp2 += x;
//         t12->fstel += x; t12->lstel += x;
//         root = merge(merge(t11, t12), t2);
// }

// int findLastG(Node* t, int x) {
//         if (t == nullptr) return -1;
```

```
//         push(t);
//         if (!t->r || t->r->mx <= x) {
//                 if (t->cur > x) return size(t->l);
//                 return findLastG(t->l, x);
//         }
//         int res = findLastG(t->r, x);
//         if (res == -1) return -1;
//         return size(t->l) + 1 + res;
// }

// int findLastL(Node* t, int x) {
//         if (t == nullptr) return -1;
//         push(t);
//         if (!t->r || t->r->mn >= x) {
//                 if (t->cur < x) return size(t->l);
//                 return findLastL(t->l, x);
//         }
//         int res = findLastL(t->r, x);
//         if (res == -1) return -1;
//         return size(t->l) + 1 + res;
// }

// int findLastHasLower(Node *t) {
//         if (t == nullptr) return -1;
//         push(t);
//         if (!t->r) {
//                 if (t->l && t->l->lstel < t->cur) {
//                         return size(t->l) - 1;
//                 }
//                 return findLastHasLower(t->l);
//         } else if (!t->r->hasLower) {
//                 if (t->cur < t->r->mx) return size(t->l);
//                 if (t->l && t->l->lstel < t->cur) return
    size(t->l) - 1;
//                 return findLastHasLower(t->l);
//         } else {
//                 return size(t->l) + 1 + findLastHasLower(t
    ->r);
//         }
// }

// int findLastHasUpper(Node *t) {
//         if (t == nullptr) return -1;
//         push(t);
//         if (!t->r) {
//                 if (t->l && t->l->lstel > t->cur) {
//                         return size(t->l) - 1;
//                 }
//                 return findLastHasUpper(t->l);
//         } else if (!t->r->hasUpper) {
//                 if (t->cur > t->r->mn) {
//                         return size(t->l);
//                 }
//                 if (t->l && t->l->lstel > t->cur) {
//                         return size(t->l) - 1;
//                 }
//                 return findLastHasUpper(t->l);
//         } else {
//                 return size(t->l) + 1 + findLastHasUpper(t
    ->r);
//         }
// }

// void next_permutation(int l, int r) {
//         auto [t1, t2] = split(root, r);
//         auto [t11, t] = split(t1, l);
//         int i = findLastHasLower(t);
//         if (i == -1) {
//                 reverse(t);
//         } else {
//                 auto [tmp, nt] = split(t, i);
//                 auto [ql, m] = split(nt, 1);
//                 int j = findLastG(m, ql->cur);
//                 auto [nm, suff] = split(m, j);
//                 auto [qr, nsuff] = split(suff, 1);
//                 nt = merge(nm, merge(ql, nsuff));
//                 reverse(nt);
//                 t = merge(tmp, merge(qr, nt));
//         }
//         root = merge(merge(t11, t), t2);
// }

// void prev_permutation(int l, int r) {
//         auto [t1, t2] = split(root, r);
//         auto [t11, t] = split(t1, l);
//         int i = findLastHasUpper(t);
//         if (i == -1) {
//                 reverse(t);
//         } else {
//                 auto [tmp, nt] = split(t, i);
//                 auto [ql, m] = split(nt, 1);
//                 int j = findLastL(m, ql->cur);
//                 auto [nm, suff] = split(m, j);
//                 auto [qr, nsuff] = split(suff, 1);
//                 nt = merge(nm, merge(ql, nsuff));
//                 reverse(nt);
//                 t = merge(tmp, merge(qr, nt));
//         }
//         root = merge(merge(t11, t), t2);
// }

} // namespace Pivo

using namespace Pivo;

void solve() {
        root = nullptr;
        int n; cin >> n;
        vi a(n);
        for (int i = 0; i < n; i++) cin >> a[i];
        for (int i = 0; i < a[n - 1]; i++) {
                push_back(i + a[n - 1]);
        }
        for (int i = a[n - 1]; i < 2 * n; i++) {
                push_back(i - a[n - 1]);
        }
        vi ans(n);
        cout << getF(root) << ' ';
        for (int i = n - 2; i >= 0; i--) {
                auto [t1, t2] = split(root, a[i]);
                auto [t21, t22] = split(t2, a[i]);
                auto tmp = t21;
                root = merge(t1, merge(t21, t22));
                auto [nt1, nt2] = split(root, 2 * n - a[i]);
                root = merge(tmp, nt1);
                cout << getF(root) << ' ';
        }
        cout << '\n';
}
```

2.6 Segment Tree

```
#include "tmp.cpp"

struct Node {
        Node() {}
};

struct SegTree {
        ve<Node> st;
        int maxn;

        SegTree() {}
        SegTree(int n) : maxn(n) {
                st.resize(4 * maxn);
        }
        SegTree(const vi& a) : maxn(a.size()) {
                st.resize(4 * maxn);
                build(a, 0, 0, maxn);
        }

        Node merge(const Node &l, const Node &r) {}

        void build(const vi& a, int v, int l, int r) {
                if (r - l == 1) {
                        // st[v] = Node(a[l]);
                        return;
                }
                int mid = (r + l) / 2;
                build(a, 2 * v + 1, l, mid);
                build(a, 2 * v + 2, mid, r);
                st[v] = merge(st[2 * v + 1], st[2 * v + 2]);
        }

        void update(int i, const Node& x, int v, int l, int r)
    {
                if (r - l == 1) {
                        st[v] = x;
                        return;
                }
                int mid = (r + l) / 2;
                if (i < mid) update(i, x, 2 * v + 1, l, mid);
                else update(i, x, 2 * v + 2, mid, r);
                st[v] = merge(st[2 * v + 1], st[2 * v + 2]);
        }

        void update(int i, int x) {
                update(i, Node(), 0, 0, maxn);
        }

        Node get(int ql, int qr, int v, int l, int r) {
                if (qr <= l || r <= ql) return Node();
                if (ql <= l && r <= qr) return st[v];
                int mid = (r + l) / 2;
                return merge(get(ql, qr, 2 * v + 1, l, mid),
    get(ql, qr, 2 * v + 2, mid, r));
        }

        Node get(int ql, int qr) {
                return get(ql, qr, 0, 0, maxn);
        }

        void buildPath(vi& path, int ql, int qr, int v, int l,
     int r) {
                if (qr <= l || r <= ql) return;
                if (ql <= l && r <= qr) {
                        path.push_back(v);
                        return;
                }
                int mid = (r + l) / 2;
                buildPath(path, ql, qr, 2 * v + 1, l, mid);
                buildPath(path, ql, qr, 2 * v + 2, mid, r);
```

```
        }

        // k behind
        int findKth(int ql, int qr, int k) {
                vi path;
                buildPath(path, ql, qr, 0, 0, maxn);
                int pref = 0;
                // for (int v : path) {
                //         if (pref + st[v].x > k) {
                //                 return st[v].ind;
                //         }
                //         pref += st[v].x;
                // }
                return -1;
        }
};

struct SegTreeMass {
        // NOTE: push внутри вершины
        ve<Node> st;
        int maxn;

        SegTreeMass() {}
        SegTreeMass(int n) : maxn(n) {
                st.resize(4 * maxn);
        }
        SegTreeMass(const vi& a) : maxn(a.size()) {
                st.resize(4 * maxn);
                build(a, 0, 0, maxn);
        }

        Node merge(const Node &l, const Node &r) {}

        void push(int v) {}

        void build(const vi& a, int v, int l, int r) {
                if (r - l == 1) {
                        // st[v] = Node(a[l]);
                        return;
                }
                int mid = (r + l) / 2;
                build(a, 2 * v + 1, l, mid);
                build(a, 2 * v + 2, mid, r);
                st[v] = merge(st[2 * v + 1], st[2 * v + 2]);
        }

        void update(int ql, int qr, const Node& x, int v, int
    l, int r) {
                if (qr <= l || r <= ql) return;
                if (ql <= l && r <= qr) {
                        st[v] = x;
                        return;
                }
                int mid = (r + l) / 2;
                push(v);
                update(ql, qr, x, 2 * v + 1, l, mid);
                update(ql, qr, x, 2 * v + 2, mid, r);
                st[v] = merge(st[2 * v + 1], st[2 * v + 2]);
        }

        void update(int ql, int qr, int x) {
                update(ql, qr, Node(), 0, 0, maxn);  // don`t
    forget set push in Node()
        }

        Node get(int ql, int qr, int v, int l, int r) {
                if (qr <= l || r <= ql) return Node();
                if (ql <= l && r <= qr) return st[v];
                int mid = (r + l) / 2;
                push(v);
                return merge(get(ql, qr, 2 * v + 1, l, mid),
    get(ql, qr, 2 * v + 2, mid, r));
        }

        Node get(int ql, int qr) {
                return get(ql, qr, 0, 0, maxn);
        }

        void buildPath(vi& path, int ql, int qr, int v, int l,
     int r) {
                if (qr <= l || r <= ql) return;
                if (ql <= l && r <= qr) {
                        path.push_back(v);
                        return;
                }
                int mid = (r + l) / 2;
                push(v);
                buildPath(path, ql, qr, 2 * v + 1, l, mid);
                buildPath(path, ql, qr, 2 * v + 2, mid, r);
        }

        // k behind
        int findKth(int ql, int qr, int k) {
                vi path;
                buildPath(path, ql, qr, 0, 0, maxn);
                int pref = 0;
                // for (int v : path) {
                //         if (pref + st[v].x > k) {
                //                 return st[v].ind;
                //         }
                //         pref += st[v].x;
                // }
                return -1;
        }
};
```

2.7 Fast Segment Tree

```
#include "tmp.cpp"

struct SegTreeFast {
        struct Node {
                int x;
                Node() : x(INFLL) {}
                Node(int _x) : x(_x) {}
        };

        Node merge(const Node &l, const Node &r) {
                return (l.x <= r.x) ? l : r;
        }

        int maxn;
        ve<Node> st;

        SegTreeFast() {}
        SegTreeFast(int n) : maxn(n) {
                st.resize(maxn * 2);
        }
        SegTreeFast(vi &a) : maxn(a.size()) {
                st.resize(2 * maxn);
                for (int i = 0; i < maxn; i++) {
                        st[maxn + i] = Node(a[i]);
                }
                for (int i = maxn - 1; i > 0; i--) {
                        st[i] = merge(st[(i << 1LL)], st[(i <<
    1LL) | 1]);
                }
        }

        void update(int i, int x) {
                i += maxn;
                st[i] = x;
                i >>= 1;
                while (i) {
                        st[i] = merge(st[(i << 1LL)], st[(i <<
    1LL) | 1]);
                        i >>= 1;
                }
        }

        Node get(int l, int r) {
                l += maxn;
                r += maxn;
                Node res;
                while (l < r) {
                        if (l & 1) {
                                res = merge(res, st[l]);
                                l++;
                        }
                        if (r & 1) {
                                r--;
                                res = merge(res, st[r]);
                        }
                        l >>= 1;
                        r >>= 1;
                }
                return res;
        }
};

struct SegTreeFastMass {
        struct Node {
                int x;
                Node() : x(INFLL) {}
                Node(int _x) : x(_x) {}
        };

        Node merge(const Node &l, const Node &r) {
                return (l.x <= r.x) ? l : r;
        }

        int maxn;
        int logn;
        ve<Node> st;
        vi sp;

        SegTreeFastMass() {}
        SegTreeFastMass(int n, int lg) : maxn(n), logn(lg) {
                st.resize(2 * maxn);
                sp.resize(maxn);
        }
        SegTreeFastMass(vi &a, int lg) : maxn(a.size()), logn(
    lg) {
                st.resize(2 * maxn);
                sp.resize(maxn);
                for (int i = 0; i < maxn; i++) st[i + maxn] =
    Node(a[i]);
                for (int i = maxn - 1; i > 0; i--) st[i] =
    merge(st[(i << 1LL)], st[(i << 1LL) | 1]);
        }

        void update(int l, int r, int x) {
```

```
                l += maxn, r += maxn;
                int ql = l, qr = r;
                while (l < r) {
                        if (l & 1) {
                                st[l].x += x;
                                if (l < maxn) sp[l] += x;
                                l++;
                        }
                        if (r & 1) {
                                r--;
                                st[r].x += x;
                                if (r < maxn) sp[r] += x;
                        }
                        l >>= 1;
                        r >>= 1;
                }
                while (ql < qr) {
                        if (ql & 1) {
                                st[ql] = merge(st[(ql << 1LL)
    ], st[(ql << 1LL) | 1]);
                                ql++;
                        }
                        if (qr & 1) {
                                qr--;
                                st[qr] = merge(st[(qr << 1LL)
    ], st[(qr << 1LL) | 1]);
                        }
                        ql >>= 1;
                        qr >>= 1;
                }
        }

        void push(int v) {
                for (int i = logn - 1; i > 0; i--) {
                        int u = (v >> i);
                        st[(u << 1LL)].x += sp[u];
                        if ((u << 1LL) < maxn) sp[(u << 1LL)]
    += sp[u];
                        st[(u << 1LL) | 1].x += sp[u];
                        if ((u << 1LL) | 1 < maxn) sp[(u << 1
    LL) | 1] += sp[u];
                        sp[u] = 0;
                        st[u] = merge(st[(u << 1LL)], st[(u <<
     1LL) | 1]);
                }
        }

        Node get(int l, int r) {
                l += maxn, r += maxn;
                push(l);
                push(r);
                Node res;
                while (l < r) {
                        if (l & 1) {
                                res = merge(res, st[l++]);
                        }
                        if (r & 1) {
                                res = merge(res, st[--r]);
                        }
                        l >>= 1;
                        r >>= 1;
                }
                return res;
        }
};
```

2.8 Splay Tree

```
#include "tmp.cpp"
#include "node.cpp"

namespace SplayTree {

void print_dfs(ve<Node *> &nodes, Node *v) {
        if (!v) return;
        v->push();
        print_dfs(nodes, v->l);
        nodes.push_back(v);
        print_dfs(nodes, v->r);
}

void print(Node *v, string s = "", bool f = true) {
        ve<Node *> nodes;
        print_dfs(nodes, v);
        if (f) cout << "{ ";
        for (auto el : nodes) {
                cout << el->x << ' ';
        }
        if (f) cout << "} " << s << "\n";
        else cout << '\n';
}

bool isRoot(Node* v) {
        return !v->p || (v->p->l != v && v->p->r != v);
}

// int chnum(Node* v) {
//         return v->p->ch[1] == v;
// }

void rotate(Node* v) {  // v -- нижний
        Node *u = v->p;
        if (u == nullptr) return;
        u->push();
        v->push();
        v->p = u->p;
        if (v->p != nullptr) {
                if (v->p->l == u) {
                        v->p->l = v;
                }
                if (v->p->r == u) {
                        v->p->r = v;
                }
        }
        if (v == u->l) {
                if (v->r) v->r->p = u;
                u->l = v->r;
                v->r = u;
        } else {
                if (v->l) v->l->p = u;
                u->r = v->l;
                v->l = u;
        }
        u->p = v;
        u->pull();
        v->pull();
}

void splay(Node *v) {
        if (v == nullptr) return;
        while (!isRoot(v)) {
                Node *u = v->p;
                if (isRoot(u)) {  // zig
                        rotate(v);
                        continue;
                }
                Node *g = u->p;
                if ((g->l == u) ^ (u->l == v)) {  // zig-zag
                        rotate(v);
                } else {
                        rotate(u);
                }
                rotate(v);
        }
}

Node * findKth(Node *t, int k) {  // without splay !!!
        if (!t) return nullptr;
        t->push();
        if (size(t->l) > k) {
                return findKth(t->l, k);
        } else if (size(t->l) == k) {
                return t;
        } else {
                return findKth(t->r, k - size(t->l) - 1);
        }
}

pr<Node *, Node *> split(Node *t, int k) { // {k els, ost}
        if (!t) return {nullptr, nullptr};
        if (k == 0) return {nullptr, t};
        Node *t1 = findKth(t, k - 1);
        splay(t1);
        Node *t2 = t1->r;
        if (t2) t2->p = nullptr;
        t1->r = nullptr;
        t1->pull();
        return {t1, t2};
}

Node * merge(Node *t1, Node *t2) { // <, >
        if (!t1) return t2;
        if (!t2) return t1;
        t1 = findKth(t1, t1->sz - 1);
        splay(t1);
        t1->r = t2;
        t2->p = t1;
        t1->push();
        t1->pull();
        return t1;
}

void insert(Node *&t, int x, int pos) {
        auto [t1, t2] = split(t, pos);
        t = new Node(x, t1, t2);
        t->pull();
}

void remove(Node *&t, int pos) {
        auto [t1, t2] = split(t, pos);
        auto [t21, t22] = split(t2, 1);
        t = merge(t1, t22);
        delete t21;
}

}  // namespace SplayTree

using namespace SplayTree;
```

2.9 Sparse Table

```
#include "tmp.cpp"

int get(const ve<vi> &spt, int l, int r) {
        int k = 31 - __builtin_clz(r - l);
        return min(spt[k][l], spt[k][r - (1 << k)]);
}

ve<vi> build(const vi &a, int n, int logn) {
        ve<vi> spt(logn, vi(n, INF));
        for (int i = 0; i < n; i++) {
                spt[0][i] = a[i];
        }
        for (int l = 0; l < logn - 1; l++) {
                for (int i = 0; i + (2 << l) <= n; i++) {
                        spt[l + 1][i] = min(spt[l][i], spt[l][
    i + (1 << (l))]);
                }
        }
        return move(spt);
}
```

2.10 Treap

```
#include "tmp.cpp"
#include "node.cpp"

mt19937 rng(1543);

uniform_int_distribution<int> dist(1, (1 << 30));

// class Node {
// public:
//         Node *l, *r;
//         int sz;
//         int y;
//         int x;
//         int summ;
//         bool reversed;

//         Node() {
//                 summ = 0;
//                 reversed = false;
//                 y = dist(rng);
//                 sz = 1;
//                 x = 0;
//                 l = nullptr;
//                 r = nullptr;
//         }

//         Node(int _x) {
//                 reversed = false;
//                 y = dist(rng);
//                 sz = 1;
//                 summ = _x;
//                 x = _x;
//                 l = nullptr;
//                 r = nullptr;
//         }

//         void push() {
//                 if (reversed) {
//                         if (l) l->reverse();
//                         if (r) r->reverse();
//                         reversed = false;
//                 }
//         }

//         void pull() {
//                 summ = x;
//                 if (l) summ += l->summ;
//                 if (r) summ += r->summ;
//                 sz = 1;
//                 if (l) sz += l->sz;
//                 if (r) sz += r->sz;
//         }

//         void reverse() {
//                 reversed ^= 1;
//                 swap(l, r);
//         }
// };

// int _size(Node *t) {
//         return (t) ? t->sz : 0;
// }

// int _summ(Node *t) {
//         return (t) ? t->summ : 0;
// }

namespace Pivo {

// к это именно размер поддерева
pr<Node*, Node*> split(Node *t, int k) {  // <= >
        if (!t) return {nullptr, nullptr};
        t->push();
        if (size(t) <= k) return {t, nullptr};
        if (k == 0) return {nullptr, t};
        if (size(t->l) >= k) {
                auto [t1, t2] = split(t->l, k);
                t->l = t2;
                t->pull();
                return {t1, t};
        } else {
                auto [t1, t2] = split(t->r, k - size(t->l) -
    1);
                t->r = t1;
                t->pull();
                return {t, t2};
        }
}

Node * merge(Node *t1, Node *t2) {
        if (!t1) return t2;
        if (!t2) return t1;
        t1->push();
        t2->push();
        if (t1->y >= t2->y) {
                t1->r = merge(t1->r, t2);
                t1->pull();
                return t1;
        } else {
                t2->l = merge(t1, t2->l);
                t2->pull();
                return t2;
        }
}

Node *root = nullptr;

void insert(int x, int pos) {
        auto [t1, t2] = split(root, pos);
        root = merge(merge(t1, new Node(x)), t2);
}

void remove(int pos) {
        auto [t1, t2] = split(root, pos);
        auto [t11, t12] = split(t2, 1);
        root = merge(t1, t12);
        delete t11;
}

int getSumm(int l, int r) {
        auto [t1, t2] = split(root, r);
        auto [t11, t12] = split(t1, l);
        if (!t12) return 0;
        int res = t12->summ;
        root = merge(merge(t11, t12), t2);
        return res;
}

// void assign(long long x, int l, int r) {
//         auto [t1, t2] = split(root, r);
//         auto [t11, t12] = split(t1, l);
//         t12->hasLower = false; t12->hasUpper = false;
//         t12->mn = x; t12->mx = x;
//         t12->summ = x * t12->sz; t12->cur = x;
//         t12->pushSet = true; t12->sp = x;
//         t12->fstel = x; t12->lstel = x;
//         t12->sp2 = 0;
//         t1 = merge(t11, t12);
//         root = merge(t1, t2);
// }

// void add(long long x, int l, int r) {
//         auto [t1, t2] = split(root, r);
//         auto [t11, t12] = split(t1, l);
//         t12->mn += x; t12->mx += x;
//         t12->summ += x * t12->sz; t12->cur += x;
//         t12->sp2 += x;
//         t12->fstel += x; t12->lstel += x;
//         root = merge(merge(t11, t12), t2);
// }

// int findLastG(Node* t, int x) {
//         if (t == nullptr) return -1;
//         push(t);
//         if (!t->r || t->r->mx <= x) {
//                 if (t->cur > x) return size(t->l);
//                 return findLastG(t->l, x);
//         }
//         int res = findLastG(t->r, x);
//         if (res == -1) return -1;
//         return size(t->l) + 1 + res;
// }

// int findLastL(Node* t, int x) {
//         if (t == nullptr) return -1;
//         push(t);
//         if (!t->r || t->r->mn >= x) {
//                 if (t->cur < x) return size(t->l);
//                 return findLastL(t->l, x);
//         }
//         int res = findLastL(t->r, x);
//         if (res == -1) return -1;
//         return size(t->l) + 1 + res;
// }

// int findLastHasLower(Node *t) {
//         if (t == nullptr) return -1;
```

```
//         push(t);
//         if (!t->r) {
//                 if (t->l && t->l->lstel < t->cur) {
//                         return size(t->l) - 1;
//                 }
//                 return findLastHasLower(t->l);
//         } else if (!t->r->hasLower) {
//                 if (t->cur < t->r->mx) return size(t->l);
//                 if (t->l && t->l->lstel < t->cur) return
    size(t->l) - 1;
//                 return findLastHasLower(t->l);
//         } else {
//                 return size(t->l) + 1 + findLastHasLower(t
    ->r);
//         }
// }

// int findLastHasUpper(Node *t) {
//         if (t == nullptr) return -1;
//         push(t);
//         if (!t->r) {
//                 if (t->l && t->l->lstel > t->cur) {
//                         return size(t->l) - 1;
//                 }
//                 return findLastHasUpper(t->l);
//         } else if (!t->r->hasUpper) {
//                 if (t->cur > t->r->mn) {
//                         return size(t->l);
//                 }
//                 if (t->l && t->l->lstel > t->cur) {
//                         return size(t->l) - 1;
//                 }
//                 return findLastHasUpper(t->l);
//         } else {
//                 return size(t->l) + 1 + findLastHasUpper(t
    ->r);
//         }
// }

// void next_permutation(int l, int r) {
//         auto [t1, t2] = split(root, r);
//         auto [t11, t] = split(t1, l);
//         int i = findLastHasLower(t);
//         if (i == -1) {
//                 reverse(t);
//         } else {
//                 auto [tmp, nt] = split(t, i);
//                 auto [ql, m] = split(nt, 1);
//                 int j = findLastG(m, ql->cur);
//                 auto [nm, suff] = split(m, j);
//                 auto [qr, nsuff] = split(suff, 1);
//                 nt = merge(nm, merge(ql, nsuff));
//                 reverse(nt);
//                 t = merge(tmp, merge(qr, nt));
//         }
//         root = merge(merge(t11, t), t2);
// }

// void prev_permutation(int l, int r) {
//         auto [t1, t2] = split(root, r);
//         auto [t11, t] = split(t1, l);
//         int i = findLastHasUpper(t);
//         if (i == -1) {
//                 reverse(t);
//         } else {
//                 auto [tmp, nt] = split(t, i);
//                 auto [ql, m] = split(nt, 1);
//                 int j = findLastL(m, ql->cur);
//                 auto [nm, suff] = split(m, j);
//                 auto [qr, nsuff] = split(suff, 1);
//                 nt = merge(nm, merge(ql, nsuff));
//                 reverse(nt);
//                 t = merge(tmp, merge(qr, nt));
//         }
//         root = merge(merge(t11, t), t2);
// }

} // namespace Pivo
```

3 Geometry

3.1 Graham Scan

```
#include "point2d.cpp"

int half(pt& p) {
    if ((p.y) > 0 || (p.y) == 0 && (p.x) >= 0) {
        return 0;
    }
    return 1;
}
bool comp(pt& l, pt& r) {
    if (half(l) < half(r))
        return 1;
    if (half(l) > half(r))
        return 0;
    if ((l % r) == 0) {
        return l.len2() < r.len2();
    }
    return (l % r) > 0;
}
    for (int i = 0; i < n; ++i)
        cin >> a[i].second >> a[i].first;
    sort(a.begin(), a.end());
    vector<pt> p;
    pt b = pt(a[0].second, a[0].first);
    for (int i = 1; i < n; ++i) {
        pt j = pt(a[i].second, a[i].first);
        j = j - b;
        p.push_back(j);
    }
    sort(p.begin(), p.end(), comp);
    vector<pt> ans;
    pt g = pt(0, 0);
    ans.push_back(g);
    ans.push_back(p[0]);
    for (int i = 1; i < n - 1; ++i) {
        while (ans.size() >= 2) {
            pt p1 = p[i] - ans.back();
            pt p2 = ans.back() - ans[(int)ans.size() - 2];
            if ((p1 % p2) >= 0)
                ans.pop_back();
            else
                break;
        }
        ans.push_back(p[i]);
    }
    cout << ans.size() << '\n';
    for (int i = 0; i < ans.size(); ++i) {
        ans[i] = ans[i] + b;
    }
```

3.2 Jarvis March

```
#include "point2d.cpp"

int maxn = 2e4 + 100;
int n;
vector<pt> a(maxn);
vector<int> used(maxn), ans(maxn);
int next(int x, int f) {
    int mn = -1;
    for (int i = 0; i < n; ++i)
        if (!used[i] || (i == f && i != x)) {
            if (mn == -1) {
                mn = i;
            } else {
                pt p1 = a[i] - a[x];
                pt p2 = a[mn] - a[x];
                if (p1 % p2 > 0) {
                    mn = i;
                } else if (p1 % p2 == 0) {
                    if (p1.len2() < p2.len2())
                        mn = i;
                }
            }
        }

    return mn;
}

    cin >> n;

    for (int i = 0; i < n; ++i)
        cin >> a[i];
    int cnt = 1;
    while (true) {
        int l = -1;
        for (int i = 0; i < n; ++i)
            if (!used[i]) {
                if (l == -1)
                    l = i;
                else {
                    if (a[i].y < a[l].y || (a[i].y == a[l].y
     && a[i].x < a[l].x))
                        l = i;
                }
            }
        if (l == -1)
            break;
        used[l] = 1;
        int f = l;
        while (true) {
            ans[l] = cnt;
            int nxt = next(l, f);
            if (nxt == f || nxt == -1)
                break;
            used[nxt] = 1;

            l = nxt;
        }
        cnt++;
    }
    for (int i = 0; i < n; ++i) {
        cout << ans[i] << '\n';
    }
```

3.3 2D Geometry

```
#include "tmp.cpp"

ld eps = 1e-7;

bool lt(ld a, ld b) {
    return b - a > eps;
}

bool eq(ld a, ld b) {
    return abs(a - b) < eps;
}

int sgn(ld a) {
    if (a > 0)
        return 1;
    if (a < 0)
        return -1;
    return 0;
}
template <typename T>
struct mpt {
    T x, y;
    mpt() {
        x = 0;
        y = 0;
    }

    mpt(T _x, T _y) {
        x = _x;
        y = _y;
    }

    mpt(mpt& a, mpt& b) {
        x = b.x - a.x;
        y = b.y - a.y;
    }

    mpt operator+(const mpt& a) const {
        return mpt(x + a.x, y + a.y);
    }

    mpt operator-(const mpt& a) const {
        return mpt(x - a.x, y - a.y);
    }

    mpt operator*(const ld k) const {
        return mpt(x * k, y * k);
    }

    bool operator==(const mpt& a) const {
        return eq(x, a.x) && eq(y, a.y);
    }

    T operator*(const mpt& a) const {
        return x * a.x + y * a.y;
    }

    T operator%(const mpt& a) const {
        return x * a.y - y * a.x;
    }

    T len2() const {
        return x * x + y * y;
    }

    ld len() const {
        return sqrt(len2());
    }

    mpt turn90() const {
        return mpt(-y, x);
    }

    ld angle(mpt& a) const {
        mpt p1 = mpt(x, y);
        return atan2l(p1 % a, p1 * a);
    }

    mpt pol() const {
        mpt a = mpt(1, 0);
        return angle(a);
    }
};

using pt = mpt<int>;

template <typename T>
struct mln {
    T a, b, c;

    mln(T _a, T _b, T _c) {
        a = _a;
        b = _b;
        c = _c;
    }

    T g(const pt& p) const {
        return a * p.x + b * p.y + c;
    }

    pt nr() const {
        return pt(a, b);
    }

    pt pr() const {
        return pt(-b, a);
    }

    mln(pt& t, pt& q) {
        pt p = pt(t, q);
        a = p.y;
        b = -p.x;
        c = -a * q.x - b * q.y;
    }

    ld dist(const pt& p) const {
        pt n = nr();
        return abs(a * p.x + b * p.y + c) / n.len();
    }

    pt proj(const pt& p) const {
        pt n = nr();
        ld d = (ld)g(p) / (ld)n.len2();
        n = p - n * d;
        return n;
    }
};

using ln = mln<ld>;
template <typename T>
struct mcr {
    pt c;
    T r;
    T x, y;
    mcr(pt& _c, T _r) {
        c = _c;
        x = c.x;
        y = c.y;
        r = _r;
    }
    array<pt, 2> fln(const ln& l) const {
        pt inf = pt(1e9, 1e9);
        ld d = l.dist(c);
        if (lt(r, d))
            return {inf, inf};
        pt p = l.proj(c);
        pt v = l.pr();
        d = r * r - d * d;
        d = sqrt(d);
        d /= v.len();
        v = v * d;
        pt p1 = p + v;
        pt p2 = p - v;
        if (eq(l.dist(c), r))
            return {p1, inf};
        return {p1, p2};
    }

    array<pt, 2> fcr(const mcr& w) const {
        if (c == w.c) {
            if (r == w.r)
                return {pt(-1e9, -1e9), pt()};
            return {pt(1e9, 1e9), pt()};
        }
        ln l = ln(2 * (x - w.x), 2 * (y - w.y), r * r - w.r *
     w.r + w.x * w.x + w.y * w.y - x * x - y * y);
        return fln(l);
    }

    array<pt, 2> tang(pt& p) {
        pt v = pt(c, p);
        ld r1 = sqrtl(v.len2() - r * r);
        mcr w = mcr(p, r1);
        return fcr(w);
    }
};

using cr = mcr<ld>;
istream& operator>>(istream& is, pt& a) {
    is >> a.x >> a.y;
    return is;
}

ostream& operator<<(ostream& is, pt& a) {
    is << a.x << ' ' << a.y;
    return is;
}

vector<pt> a;

bool f(pt p, pt l, pt r) {
    return sgn(p % l) != sgn(p % r) || sgn(p % l) == 0 || sgn(
     p % r) == 0;
}

void mincockskii(vi st) {
    int n = st.size();
    for (int i = 0; i < n; ++i) {
        st[i] = st[i] + upp;
    }
    vector<pt> p1(n);
```

```
    for (int i = 0; i < n; ++i) {
        p1[i] = pt(-st[i].x, -st[i].y);
    }
    int ij = 0;
    for (int i = 0; i < n; ++i) {
        if (comp1(p1[i], p1[ij])) {
            ij = i;
        }
    }
    vector<pt> st1;
    for (int cnt = 0; cnt < n; cnt++, ij++, ij %= n) {
        st1.push_back(p1[ij]);
    }
        p = st;
    p1 = st1;
    int l = 1, r = 1;
    pt rt = p[0] + p1[0];
    int f1 = 0, f2 = 0;
    int cnt = 0;
    vector<array<ld, 2> > gt;
    while (f1 + f2 < 2 && cnt < 2 * n) {
        pt u1 = p[l] - p[(l - 1 + n) % n];
        pt u2 = p1[r] - p1[(r - 1 + n) % n];
        if (f1 == 0 && u1 % u2 > 0) {
            rt = rt + u1;
            gt.push_back({rt.x * rt.x, rt.y * rt.y});
            l++;
            l %= n;
            cnt++;
            if (l == 1) {
                f1 = 1;
            }
        } else {
            rt = rt + u2;
            gt.push_back({rt.x * rt.x, rt.y * rt.y});
            r++;
            cnt++;
            r %= n;
            if (r == 1) {
                f2 = 1;
            }
        }
    }
    sort(gt.begin(), gt.end());
}

int32_t main() {
    int n, m, k;
    cin >> n >> m >> k;
    a.resize(n);
    for (int i = 0; i < n; ++i)
        cin >> a[i];
    while (m--) {
        pt p;
        cin >> p;
        int l = 1;
        int r = n - 1;
        if (f(p - a[0], a[l] - a[0], a[r] - a[0])) {
            while (r - l > 1) {
                int x = (l + r) / 2;
                if (f(p - a[0], a[l] - a[0], a[x] - a[0]))
                    r = x;
                else
                    l = x;
            }
            pt u = pt(a[l], a[l + 1]);
            pt v = pt(a[l], p);
            if (f(v, u, a[0] - a[l]) && f(p - a[l + 1], a[0] -
     a[l + 1], a[l] - a[l + 1]))
                k--;
        }
    }
    if (k <= 0) {
        cout << "YES\n";
    } else {
        cout << "NO\n";
    }
    return 0;
}
```

4 Graphs

4.1 Centroid Decomposition

```
#include "tmp.cpp"

// TODO: проверить

ve<vi> g;
vi P;
vi sz;
vi removed;

void sizes(int v, int p) {
    sz[v] = 1;
    for (auto u : g[v]) {
        if (u != p && !removed[u]) {
            sizes(u, v);
            sz[v] += sz[u];
        }
    }
}

int find(int v, int p, int n) {
    for (auto u : g[v]) {
        if (u != p && !removed[u] && sz[u] > n / 2) {
            return find(u, v, n);
        }
    }
    return v;
}

ve<map<int, int>> dist;
vi a;

void update(int v, int p, int c, int d) {
    if (dist[c][a[v]] == 0 || dist[c][a[v]] > d) {
        dist[c][a[v]] = d;
    }
    for (auto u : g[v]) {
        if (!removed[u] && u != p) {
            update(u, v, c, d + 1);
        }
    }
}

void decompose(int v, int p) {
    sizes(v, -1);
    int c = find(v, -1, sz[v]);
    P[c] = p;
    removed[c] = true;
    update(c, -1, c, 0);

    for (auto u : g[c]) {
        if (!removed[u]) decompose(u, c);
    }
}

ve<vi> up;
vi tin, tout, dp;
const int LOGN = 30;
int T = 0;

void dfs(int v, int p, int d) {
    tin[v] = T++;
    for (int l = 1; l < LOGN; l++) {
        up[v][l] = up[up[v][l - 1]][l - 1];
    }
    dp[v] = d;
    for (auto u : g[v]) {
        if (u == p) continue;
        up[u][0] = v;
        dfs(u, v, d + 1);
    }
    tout[v] = T;
}

bool is_pred(int v, int u) {
    return tin[v] <= tin[u] && tout[v] >= tout[u];
}

int count_dist(int v, int u) {
    if (is_pred(v, u)) return dp[u] - dp[v];
    if (is_pred(u, v)) return dp[v] - dp[u];
    int ast = v;
    for (int l = LOGN - 1; l >= 0; l--) {
        if (!is_pred(up[v][l], u)) {
            v = up[v][l];
        }
    }
    return dp[ast] + dp[u] - dp[up[v][0]] * 2;
}

int get(int v, int c) {
    // cerr << v << ' ' << c << '\n';
    if (a[v] == c) return 0;
    int res = INF;
    int sv = v;
    // cerr << 0 << '\n';
    while (v != -1) {
        // cerr << 1 << '\n';
        int cd = count_dist(v, sv);
        // cerr << 2 << '\n';
        if (dist[v][c] != 0 || a[v] == c) {
            res = min(res, dist[v][c] + cd);
        }
        // cout << v << ' ' << cd << ' ' << res << '\n';
        // cerr << 3 << '\n';
        v = P[v];
    }
    return (res == INF) ? -1 : res;
}
```

4.2 Cut Points

```
#include "tmp.cpp"
```

```
set<int> pts;
vi used;
vi up;
vi h;
ve<vi> g;

void dfs(int v, int p = -1) {
        up[v] = h[v];
        used[v] = true;
        int cnt = 0;
        int ch = 0;
        for (auto u : g[v]) {
                if (u == v) continue;
                if (u == p && !cnt) {
                        cnt++;
                        continue;
                }
                if (used[u]) fchmin(up[v], h[u]);
                else {
                        ch++;
                        h[u] = h[v] + 1;
                        dfs(u, v);
                        fchmin(up[v], up[u]);
                        if (up[u] >= h[v] && p != -1) {
                                pts.insert(v);
                        }
                }
        }
        if (p == -1 && ch > 1) {
                pts.insert(v);
        }
}
```

4.3 Heavy-Light Decomposition

5 Optimizations and Transforms

5.1 Convex Hull Trick

```
// КХТ на максимум


#include "tmp.cpp"

struct Line {
        int k, b;
        int operator()(int x) const {
                return k * x + b;
        }
        Line(int _k, int _b) : k(_k), b(_b) {}
        Line() : k(0), b(0) {}
};

void cht() {
        auto cross = [ & ](const Line& l1, const Line& l2) ->
    int {
                if (l1.k == l2.k) return -INF;
                if (l2.b < l1.b) {
                        return (l2.b - l1.b) / (l1.k - l2.k);
                }
                return (l2.b - l1.b + (l1.k - l2.k - 1)) / (l1
    .k - l2.k);
        };

        auto add = [ & ](ve<Line>& cht, vi &curPts, const Line
    & l) -> void {
                while (!cht.empty()) {
                        int x = cross(l, cht.back());
                        if (x <= curPts.back()) {
                                curPts.pop_back();
                                cht.pop_back();
                        } else {
                                break;
                        }
                }
                if (!cht.empty()) curPts.push_back(cross(l,
    cht.back()));
                else curPts.push_back(-INF);
                cht.push_back(l);
        };

        auto get = [ & ](ve<Line>& cht, vi &curPts, int x) ->
    pii {
                int ind = upper_bound(all(curPts), x) - curPts
    .begin() - 1;
                return {cht[ind](x), cht[ind].k};
        };
}
```

5.2 Fast Fourier Transform

```
#include "tmp.cpp"

using cd = complex<ld>;
using vcd = vector<cd>;
const ld PI = acosl(-1);

void fft(vcd &a, vcd &res, int start, int step, int n, int l,
    bool inv=false) {
        if (n == 1) {
                res[l] = a[start];
                return;
        }
        int m = n / 2;
        cd wi(1);
        ld angle = 2 * PI / n * (inv ? -1 : 1);
        cd w(cosl(angle), sinl(angle));
        fft(a, res, start, step * 2, n / 2, l, inv);
        fft(a, res, start + step, step * 2, n / 2, l + m, inv)
    ;
        for (int i = l; i < l + m; i++) {
                cd v0 = res[i];
                cd v1 = res[i + m];
                res[i] = v0 + wi * v1;
                res[i + m] = v0 - wi * v1;
                if (inv) {
                        res[i] /= 2;
                        res[i + m] /= 2;
                }
                wi *= w;
        }
}

void solve() {
        string a, b; cin >> a >> b;
        int n1 = a.length();
        int n2 = b.length();
        if (a[0] == '0' || b[0] == '0') {
                cout << 0 << '\n';
                return;
        }
        bool f1 = a[0] == '-';
        bool f2 = b[0] == '-';
        vcd A, B;
        for (int i = n1 - 1; i >= (int)f1; i--) A.push_back(a[
    i] - '0');
        for (int i = n2 - 1; i >= (int)f2; i--) B.push_back(b[
    i] - '0');
        n1 -= f1;
        n2 -= f2;
        int n = n1 + n2;
        while (n1 < n) {
                A.push_back(0);
                n1++;
        }
        while (n > n2) {
                B.push_back(0);
                n2++;
        }
        while (__builtin_popcount(n) > 1) {
                A.push_back(0);
                B.push_back(0);
                n++;
        }
        vcd r1(n), r2(n);
        fft(A, r1, 0, 1, n, 0);
        fft(B, r2, 0, 1, n, 0);
        vcd r(n);
        for (int i = 0; i < n; i++) {
                r[i] = r1[i] * r2[i];
        }
        fft(r, A, 0, 1, n, 0, true);
        if (f1 ^ f2) {
                cout << '-';
        }
        ll nakop = 0;
        vi ans;
        ans.reserve(n);
        for (int i = 0; i < n; i++) {
                nakop += llround(A[i].real());
                ans.push_back(nakop % 10);
                nakop /= 10;
        }
        while (nakop) {
                cout << nakop % 10;
                nakop /= 10;
        }
        int ind = (int)ans.size() - 1;
        while (ans[ind] == 0) ind--;
        for (; ind >= 0; ind--) cout << ans[ind];
        cout << '\n';
}
```

5.3 Compiler Optimizations

```
#include "tmp.cpp"

void solve() {
        int n; cin >> n;
        int N = (1 << n);
        vi p(N);
        for (int i = 0; i < N; i++) cin >> p[i];
        vi f = p;
        for (int i = 0; i < n; i++) {
                for (int mask = 0; mask < N; mask++) {
```

```
                    if (mask & (1 << i)) f[mask] -= f[mask
    ^ (1 << i)];
            }
      }
      for (auto& el : f) cout << el << ' ';
      cout << '\n';
}
```

5.4 XOR Convolution

```
#include "tmp.cpp"

void xoraboba(vll &a, vll &b, vll &res, int l, int r) {
  if (r - l == 1) {
    res[l] = a[l] * b[l];
    return;
  }
       int mid = (r - l) / 2;
  for (int i = l; i < l + mid; i++) {
              ll x = a[i];
              ll y = a[i + mid];
    a[i] = x + y;
    a[i + mid] = x - y;
              x = b[i];
              y = b[i + mid];
    b[i] = x + y;
    b[i + mid] = x - y;
  }
  xoraboba(a, b, res, l, l + mid);
  xoraboba(a, b, res, l + mid, r);
  for (int i = l; i < l + mid; i++) {
              ll x = res[i];
              ll y = res[i + mid];
    res[i] = (x + y) / 2;
    res[i + mid] = (x - y) / 2;
  }
}
```

6 Strings

6.1 String Algorithms

```
#include "tmp.cpp"

vi z_func(const string &s) {
       int n = s.length();
  vi z(n);
  int l = 0, r = 0;
  for (int i = 1; i < n; i++) {
    if (i < r) z[i] = min(r - i, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
      z[i]++;
    }
    if (i + z[i] > r) {
      l = i;
      r = i + z[i];
    }
  }
       return std::move(z);
}

vi pref_func(const string &s) {
       int n = s.length();
       vi p(n);
       for (int i = 1; i < n; i++) {
              int j = p[i - 1];
              while (j > 0 && s[j] != s[i]) {
                     j = p[j - 1];
              }
              if (s[i] == s[j]) j++;
              p[i] = j;
       }
       return std::move(p);
}
```